

CODELANGID: A CHARACTER-LEVEL CNN FOR PROGRAMMING LANGUAGE IDENTIFICATION

Mohamad Salman | mohamad.salman@mail.utoronto.ca | Solo project

Code: <https://github.com/mohamadmsalman82/codelangid>

1 BRIEF PROJECT DESCRIPTION

Syntax highlighters, code search engines, snippet managers and Q&A sites such as Stack Overflow must constantly identify a block of code’s language, often from a short fragment with no filename or extension (Alreshedy et al., 2018). **This project maps a raw snippet to one of 10 popular languages (Python, Java, C++, JavaScript, Go, C, Ruby, PHP, Rust, C#) from its characters alone.** Extensions and editor metadata are exactly what is missing in pasted snippets, chat messages and scraped pages; rule-based detectors break on short fragments (van Dam & Zaytsev, 2016), and GitHub’s Linguist leans on extensions (GitHub, 2025), so neither says much about a bare snippet—the gap a learned model fills.

Deep learning fits this input/output pair for three reasons. Languages differ by *subtle, overlapping* cues (keywords, operator spellings, bracket and indentation patterns) a character-level CNN learns automatically rather than by hand-engineering features per language (Zhang et al., 2015); those cues are *local* n-gram motifs (`:=`, `fn`, `$var`), which is exactly what small-kernel convolution detects (Kim, 2014); and character input side-steps a chicken-and-egg problem—a token-based model needs a tokenizer, but choosing one presupposes knowing the language. Figure 1 is the contract: **input** raw text padded/truncated to 256 characters, **output** a softmax over the 10 languages.

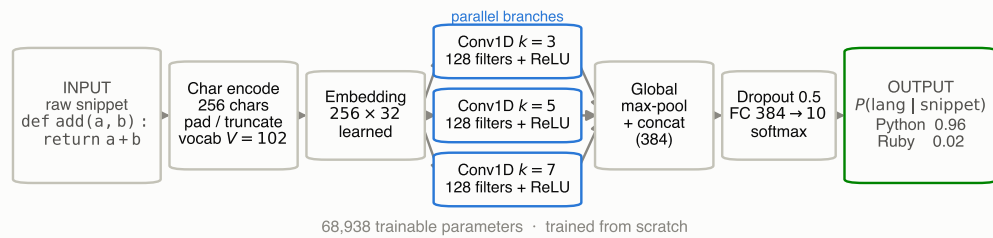


Figure 1: The CodeLangID pipeline. **Input:** a raw snippet, any language, encoded character-by-character into a fixed 256-token window. **Output:** $P(\text{lang} \mid \text{snippet})$ over the 10 classes.

2 INDIVIDUAL CONTRIBUTIONS AND RESPONSIBILITIES

This is a **solo project**: there are no other team members, so every item in Table 1 is my own work and I hold all responsibilities end to end. All code is reproducible (`src/`, seed 360); every number regenerates from a clean checkout, the CNN to within MPS non-determinism (§3.4).

Table 1: Work breakdown; every item is my sole responsibility (solo project). Only final tuning and the report remain in progress.

Component (all complete)	Evidence / artefact
Data collection & cleaning	<code>build.dataset.py</code> : 10,360 snippets (Table 2)
Held-out test collection	<code>scrape.heldout.py</code> : 740 snippets, 24 repos, 0 leaked
Baseline model	<code>baseline.py</code> : 90.14% test (Table 3)
Primary model (char-CNN)	<code>cnn.py</code> : 92.74% test, 82.84% held-out
Alternative model (BiGRU)	<code>gru.py</code> : tried and rejected (§3.3)
Evaluation, ablation & errors	<code>figures.py</code> , <code>leakage-experiment.py</code> : §3.4

3 NOTABLE CONTRIBUTION

My notable contribution is the **complete working pipeline**: a cleaned balanced dataset, a classical baseline, a char-CNN that beats it, and an independently collected held-out set exposing a diagnosable failure mode.

3.1 DATA PROCESSING

Source. Rosetta Code (ros, 2026) (acmeism/RosettaCodeData mirror) implements the same tasks across many languages, so snippets are labelled by directory, not guessed. Table 2 gives the full pipeline (`src/build_dataset.py`, seed 360) and its yield, specified so a classmate can reproduce it. Two steps carry most of the weight: *near-duplicate removal* (MinHash/LSH over 5-character shingles drops pairs with Jaccard ≥ 0.8 (Broder, 1997)—Rosetta hosts many near-identical variants of one solution) and *splitting by task, not by file*. Snippets are emitted as `raw` and—where they still clear the 64-character floor—`nocomment` (10,271), to test reliance on comment prose versus code structure.

Table 2: Cleaning pipeline and yield (steps 2–4 run during windowing, so the running total is first well-defined at step 4). Result: 10,360 snippets, 1,036 per class (chance = 10%), split 1,162/249/250 *tasks*.

#	Operation	Removed	Remaining
1	Read 10 language directories	—	18,654 files
2	Window: ≤ 3 line-aligned 256-char windows/file (≥ 3 lines, ≥ 64 char)	1,125 files	
3	Comment/markup filter: drop if comments $> 50\%$ of non-space chars	3,460 win.	
4	Exact dedup: SHA-1 of whitespace-normalized text	327 win.	38,394 snippets
5	Near-dup dedup: MinHash (64 perm, 16 LSH bands), Jaccard ≥ 0.8	353	38,041
6	Balance: cap each class at 1,036 = smallest class (PHP)	27,681	10,360
7	Split by task 70/15/15; encode to vocab $V=102$, pad/truncate to 256	—	7,225 / 1,634 / 1,501

Example cleaned training sample, verbatim from `raw_train.jsonl` (class PYTHON, task Call-a-foreign-language-function, 121 characters; the two `#` comments are what the `nocomment` variant strips):

```
import ctypes
libc = ctypes.CDLL("/lib/libc.so.6")
libc.strcmp("abc", "def") # -1
libc.strcmp("hello", "hello") # 0
```

Plan for never-before-seen data. Rather than reuse a public dataset I collected the final test set myself (`src/scrape_heldout.py`): **740 snippets from 24 real GitHub repositories**, 74 per language, through the same windowing, comment and exact-dedup steps (near-dup dedup is moot here—no two library files are variants of one task). None is a Rosetta source, and it is deliberately a *different distribution*: production library code, not puzzle solutions. Each snippet is checked against the SHA-1 of every training snippet—**0 leaked**—and is never trained or tuned on.

Challenges. (i) PHP is thin on Rosetta (645 files vs. Python’s 3,054), so balancing drags *every* class to 1,036—the biggest limit on dataset size. (ii) I split by task expecting a per-snippet split to leak; measuring it (`src/leakage_experiment.py`) showed otherwise—it moves the baseline +0.20 and the CNN +0.00, so near-dup removal, not the split, is what prevents leakage here. I kept the by-task split as the safer default. (iii) The line-based stripper over-strips string literals containing `//` or `#`, and for C deletes `#include/#define`, which are *code*—biasing the ablation *against* the CNN’s 92.49, so that conclusion is conservative. Fixed for the final.

3.2 BASELINE MODEL

The baseline uses no deep learning: TF-IDF over **character n-grams** ($n \in [1, 3]$, `min_df=2`, sublinear tf, case preserved) into **Multinomial Naive Bayes** on the same splits (Khasnabish et al., 2014; Ugurel et al., 2002). It needs no tuning beyond the fixed n-gram range, uses 43,573 features and trains in ≈ 4 s. It is an honest bar, not a straw man—**90.14%** versus 10% chance—so n-grams already carry most of the signal.

Qualitative (baseline). Its held-out errors are prose-driven: it labels PHP C# **34/74** times and gets PHP right only **26/74**—a pair it never confuses in-distribution (0 cases)—and Java→Python (24/74) and C→Python (20/74) repeat the shape: English comment blocks drag an n-gram model toward whichever class carries the most prose. **Challenge:** no tuning fixes this—the failure is the representation, which is the gap the CNN closes.

3.3 PRIMARY MODEL

The primary model (Figure 1) is Embedding(102,32) (`padding_idx=0`); three *parallel* Conv1d branches of widths 3/5/7, 128 filters each + ReLU; global max-pool per branch; concatenation to 384; Dropout(0.5); Linear(384,10) + softmax. Parallel widths rather than a deep stack see 3-, 5- and 7-character motifs at once—the scale at which `fn`, `:=` and `$var` live. It has

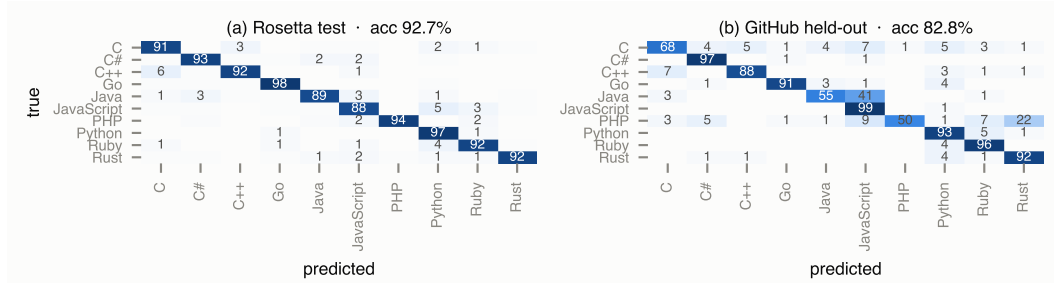


Figure 2: Char-CNN confusion, row-normalized (cell = % of the true class predicted as that column). (a) Rosetta test. (b) Never-before-seen GitHub held-out, where Java→JavaScript and PHP→Rust dominate (§3.4).

68,938 trainable parameters in 5 weight layers, trained from scratch with cross-entropy and Adam (Kingma & Ba, 2015) (lr 10^{-3} , batch 64, ≤ 30 epochs, early stopping on validation accuracy, patience 6), in **35 s** on an Apple M-series GPU (MPS). Validation loss flattens near epoch 10 and the train/validation gap widens after epoch 12—mild overfitting early stopping controls (stopped at 25, best **epoch 19**).

I also built the proposal’s bidirectional GRU (src/gru.py; 42,186 parameters). **It did not pan out**—the most useful negative result so far: 90.27% test, indistinguishable from Naive Bayes, for **647 s of training, 18× the CNN**. Language identity in a 256-character window rests on *local* motifs, so recurrence buys nothing.

3.4 QUANTITATIVE AND QUALITATIVE RESULTS

Quantitative. The char-CNN wins on every split (Table 3): **92.74%** (macro-F1 0.927, loss 0.238) on Rosetta test, **+2.60** over the baseline, and **82.84%** (macro-F1 0.822, loss 0.495) held-out, **+3.24** over it.

Table 3: All models, both test sets (accuracy %). “Held-out” = never-before-seen GitHub data. Chance 10%. Best test / held-out in bold; ft = n-gram features, not learned parameters. CNN rows are the committed seeded run (§3.4).

Model	Variant	Params	Val	Test (Rosetta)	Held-out (GitHub)
TF-IDF + NB (baseline)	raw	43,573 ft	91.80	90.14	79.59
TF-IDF + NB (baseline)	nocomment	41,472 ft	91.48	89.27	81.89
Char-BiGRU (rejected)	raw	42,186	92.59	90.27	80.54
Char-CNN (primary)	raw	68,938	94.31	92.74	82.84
Char-CNN (primary)	nocomment	68,938	94.69	92.49	82.16

Comment ablation. Stripping comments costs the CNN only **0.25** (92.74→92.49): it reads *code structure, not comment prose*. The baseline behaves oppositely—comments removed, its held-out score *rises* 2.30 (79.59→81.89).

Qualitative (CNN). The 9.9-point gap between the two test sets (Figure 2) is the most interesting result so far, because the errors explain it. (1) *Java→JavaScript* (30 errors): *licence headers*. **22 of the 30** carry an Apache/copyright header, averaging only **17% code** by non-whitespace character—the rest is English prose in a `/* */` block. Rosetta files have none, so the model never learned that a real file’s first 256 characters are often language-identical boilerplate: it is starved of code, not wrong. (2) *PHP→Rust* (16 errors): *PHP 8 attributes*. **13 of the 16** contain `# [`, diagnosable straight from training data, where it occurs in **0.0% of PHP snippets (0/753)** but **9.0% of Rust (63/703)**—so the model learned `# [⇒Rust`. PHP 8 spells attributes identically (`# [Group('...')]`) and they appear in **18.9% of held-out PHP (14/74)**, but Rosetta’s PHP predates PHP 8. Textbook *distribution shift*—confidently wrong for a legible reason—and PHP’s F1 falls 0.97→0.66.

Challenges. (i) The held-out set took the most work: my first version yielded only 34 snippets/class because the C repositories I picked were too small, so I widened the list, and the BiGRU’s 18× cost made searching over it impractical. (ii) MPS is non-deterministic, so a fixed seed does not pin the result: eight reruns span **91.0–92.7** on test (mean 92.2). Table 3 is the committed run, but the CNN beat the baseline **8/8** (worst +0.87)—the ranking is robust, the exact figure is not. **On feasibility:** all data is obtained, the baseline is beaten on both test sets, and the pipeline reruns in under 3 minutes, so the remaining work—licence-header stripping, post-2015 PHP sources, SCC (Alreshedy et al., 2018)—is comfortably achievable.

REFERENCES

- Rosetta code. <https://rosettacode.org>, 2026. Corpus mirror: <https://github.com/acmeism/RosettaCodeData>. Accessed: 2026-07-15.
- Kamel Alreshedy, Dhanush Dharmaretnam, Daniel M. German, Venkatesh Srinivasan, and T. Aaron Gulliver. SCC: Automatic classification of code snippets. *arXiv preprint arXiv:1809.07945*, 2018.
- Andrei Z. Broder. On the resemblance and containment of documents. In *Proceedings of the Compression and Complexity of Sequences*, pp. 21–29. IEEE, 1997.
- GitHub. GitHub linguist. <https://github.com/github-linguist/linguist>, 2025. Accessed: 2026-07-15.
- Jyotiska Nath Khasnabish, Mitali Sodhi, Jayati Deshmukh, and Gopalakrishnan Srinivasaraghavan. Detecting programming language from source code using bayesian learning techniques. In *Machine Learning and Data Mining in Pattern Recognition (MLDM)*, pp. 513–522. Springer, 2014.
- Yoon Kim. Convolutional neural networks for sentence classification. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 1746–1751, 2014.
- Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.
- Secil Ugurel, Robert Krovetz, and C. Lee Giles. What’s the code? automatic classification of source code archives. In *Proceedings of the 8th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 632–638, 2002.
- Juriaan Kennedy van Dam and Vadim Zaytsev. Software language identification with natural language classifiers. In *IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, pp. 624–628, 2016.
- Xiang Zhang, Junbo Zhao, and Yann LeCun. Character-level convolutional networks for text classification. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 28, 2015.